QWEN

QWEN.md — Qwen Code context for this module

This file is read by Qwen Code as its module-context file. It is the Qwen Code counterpart of CLAUDE.md and AGENTS.md for this module, and it is a pointer: there is one canonical agent-instruction file per scope.

Read CLAUDE.md — it is mandatory

This module's canonical agent-instruction file is CLAUDE.md in this directory. Before doing any work in this module, open and read CLAUDE.md and this module's CONSTITUTION.md in full. Every rule there binds Qwen Code exactly as it binds Claude Code.

This file is a plain-text pointer and deliberately uses no auto-import directive. Qwen Code's memory-import processor resolves import-prefixed tokens recursively, and the instruction files reference tokens that are not files. To stay compatible with Qwen Code this file contains no such tokens — read CLAUDE.md directly.

INHERITED FROM constitution/CLAUDE.md

This module's CLAUDE.md inherits, unconditionally, every rule in constitution/CLAUDE.md and the constitution/Constitution.md it references — the HelixConstitution submodule mounted at the parent project's constitution/ directory (resolve the path with constitution/find_constitution.sh from the parent project root). Qwen Code MUST NOT weaken any inherited rule.

MANDATORY ANTI-BLUFF END-USER-QUALITY COVENANT

Forensic anchor — verbatim user mandate (2026-04-28, reasserted 2026-05-21):

> "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

Operative rule. Tests and Challenges exist for exactly one purpose: to confirm a feature genuinely works for a real end user, end-to-end. The bar for shipping is **not** "tests pass" but **"end users can actually use the feature."** A test that passes while the feature is broken is a bluff test and is forbidden. Metadata-only PASS, configuration-only PASS, "absence-of-error" PASS, and grep-without- runtime PASS are all critical defects regardless of how green the summary line looks. CI green is necessary, never sufficient. Tests and HelixQA Challenges are bound EQUALLY.

This covenant is restated verbatim in every governance file at the consumer layer (CLAUDE.md, AGENTS.md, QWEN.md, CONSTITUTION.md) so any tool that does not expand @imports still reads it. The same verbatim block lives upstream at `constitution/Constitution.md` §11.4. See this module's CLAUDE.md, AGENTS.md, and CONSTITUTION.md for the full Sixth/Seventh Law and section 6.J / 6.L mandate.

§11.4.78 — CodeGraph code-intelligence mandate

Inherited by §11.4.78 ID reference from `constitution/Constitution.md` §11.4.78 (this module's `CLAUDE.md` and `CONSTITUTION.md` carry the full anchor with the package name and install commands). In brief: every project worked on by AI coding agents MUST install, initialize, and use CodeGraph — a local semantic code-knowledge-graph exposed to agents over MCP — wired into every CLI agent the developers use, covered by an anti-bluff verification suite. See `CLAUDE.md` and `CONSTITUTION.md` in this module, and the constitution submodule `Constitution.md` §11.4.78, for the full mandate.

§11.4.81 — Cross-platform-parity mandate

Inherited by §11.4.81 ID reference from `constitution/Constitution.md` §11.4.81 (User mandate, 2026-05-21). Every consuming project whose supported-platforms manifest lists more than one OS MUST, for every feature/test/gate/challenge/mutation that depends on platform-specific primitives, ship a per-OS-equivalent implementation chosen at runtime via `uname -s` (or equivalent platform detection). Three sub-mandates: **(A)** per-OS implementation REQUIRED via runtime dispatch (POSIX `setrlimit`/`ulimit` on macOS, launchd, BSD `rctl`, Windows Job Object); **(B)** per-OS tests REQUIRED with positive captured evidence per branch; **(C)** honest kernel-gap citation + adjacent equivalent test REQUIRED where no equivalent exists (canonical: XNU does NOT enforce `RLIMIT_AS` for unprivileged processes → SKIP with reproducer + adjacent test of what IS enforced, e.g. `RLIMIT_CPU+SIGXCPU`). The adjacent test is itself anti-bluff per §11.4 with a paired §1.1 mutation. See `CLAUDE.md`, `AGENTS.md`, `CONSTITUTION.md` for §11.4.81 in full; canonical authority at the constitution submodule `Constitution.md` §11.4.81.